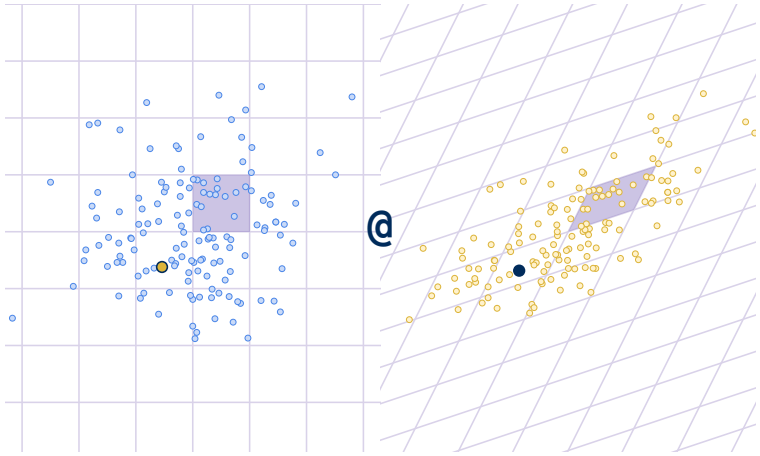
UNIT 01 · CHAPTER 1

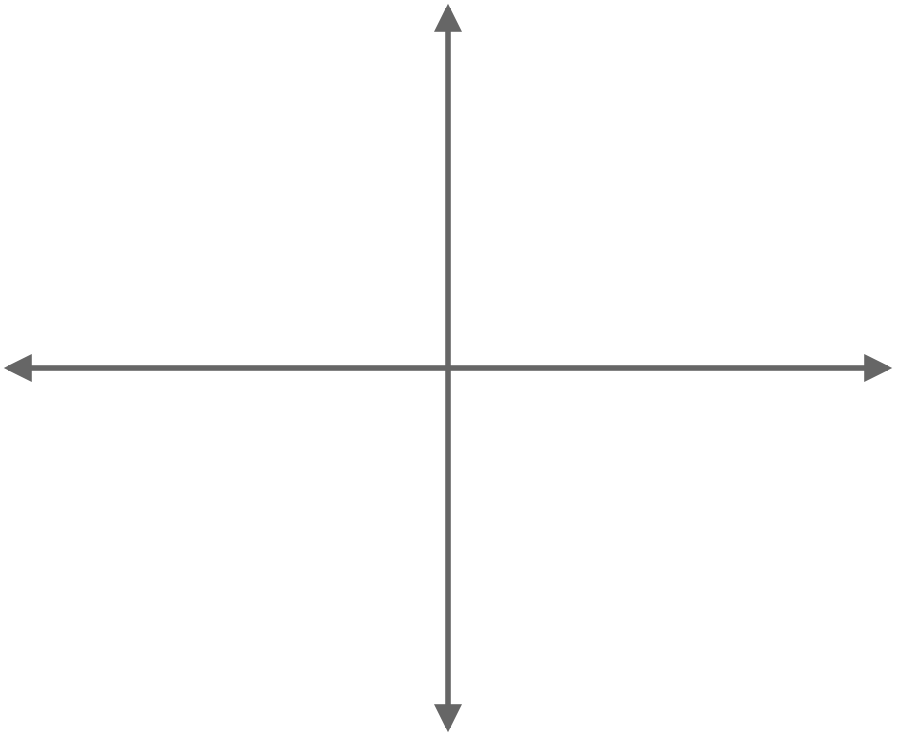
High Dimensional Data



CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

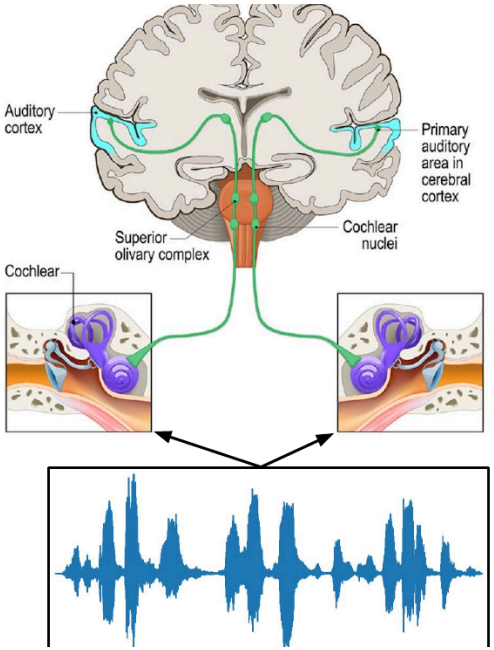

Paul Graham @pau... · 1d
Someone asked what I'd do if I were 17. I'd learn how to build LLMs from scratch, and then train ones as powerful as I could with whatever hardware I could get access to.
543
945
13K

Last Class: High Dimensional Data



Feature: one measured property

Representations in the Brain



"Which word will be heard?"

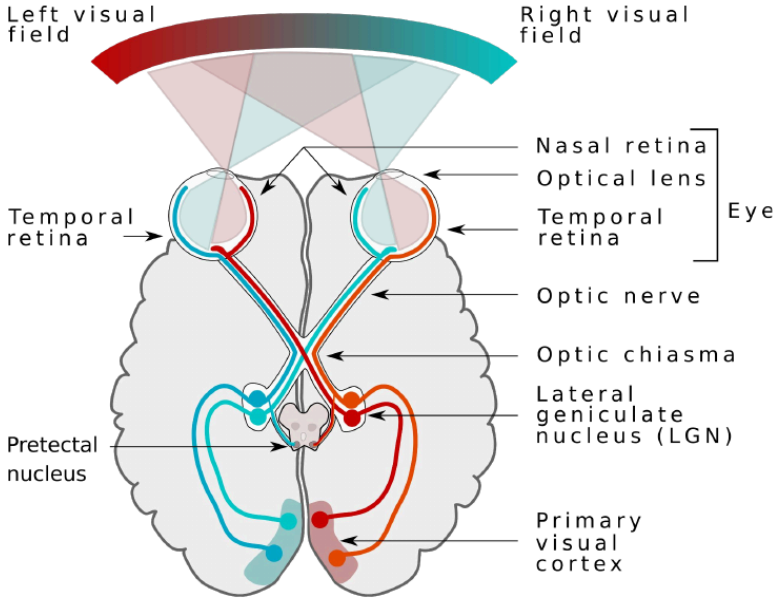
Representation of the World



Which **blue** shape is **correct**?

Representations in the Brain





Representation of the World



What **features** could we use to describe these shapes?

Notre Dame Football Dataset: HW01

Opponent	ND home	Neutral	Attendance	ND pts	Opp. pts	Margin	Win
Alabama	0	1	80120	14	42	−28	0
Purdue	1	0	80795	20	17	3	1
Mich. St.	0	0	79219	20	3	17	1
Michigan	1	0	80795	13	6	7	1
Stanford	1	0	80975	20	13	7	1
BYU	1	0	80795	17	14	3	1
Oklahoma	0	0	86031	30	13	17	1

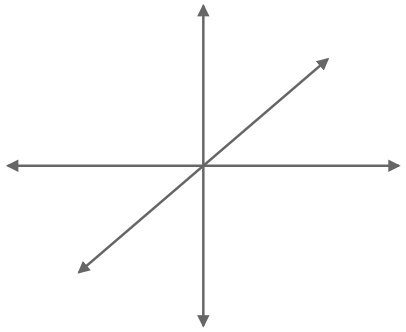
Feature vector: the ordered list of feature values for one data point.

Data point (Sample): one represented item, here one football game, often called a sample.

Dimensionality: how many features a sample has.

High Dimensional Data

Opponent	ND home	Neutral	Attendance	ND pts	Opp. pts	Margin	Win
Alabama	0	1	80120	14	42	−28	0
Purdue	1	0	80795	20	17	3	1
Mich. St.	0	0	79219	20	3	17	1
Michigan	1	0	80795	13	6	7	1
Stanford	1	0	80975	20	13	7	1
BYU	1	0	80795	17	14	3	1
Oklahoma	0	0	86031	30	13	17	1



Task: what we are trying to achieve with our model

Target: the output a model is trying to predict.

Supervised Learning

We call it **supervised learning** when we have examples of the **target** in our **dataset**, because **we can tell our model if it guessed correctly or not**. We call these correct answers **ground truth**

Opponent	ND home	Neutral	Attendance	ND pts	Opp. pts	Margin	Win
Alabama	0	1	80120	14	42	−28	0
Purdue	1	0	80795	20	17	3	1
Mich. St.	0	0	79219	20	3	17	1
Michigan	1	0	80795	13	6	7	1
Stanford	1	0	80975	20	13	7	1
BYU	1	0	80795	17	14	3	1
Oklahoma	0	0	86031	30	13	17	1

Task: Classification

In **machine learning**, a **classification** task is one in which we're trying to **predict a category or label (target)** given a **sample**.

Opponent	ND home	Neutral	Attendance	ND pts	Opp. pts	Margin	Win
Alabama	0	1	80120	14	42	−28	0
Purdue	1	0	80795	20	17	3	1
Mich. St.	0	0	79219	20	3	17	1
Michigan	1	0	80795	13	6	7	1
Stanford	1	0	80975	20	13	7	1
BYU	1	0	80795	17	14	3	1
Oklahoma	0	0	86031	30	13	17	1

For our football **dataset**, our labels **(targets)** are: `ND_Wins` and `ND_Loses`

High Dimensional Data

Opponent	ND home	Neutral	Attendance	ND pts	Opp. pts	Margin	Win
Alabama	0	1	80120	14	42	−28	0
Purdue	1	0	80795	20	17	3	1
Mich. St.	0	0	79219	20	3	17	1
Michigan	1	0	80795	13	6	7	1
Stanford	1	0	80975	20	13	7	1
BYU	1	0	80795	17	14	3	1
Oklahoma	0	0	86031	30	13	17	1

Dataset: the collection of all your feature vectors.

Numerical Feature: a measured property represented as a number.

Binary Feature: a feature with two possible values, often encoded as `0` and `1`.

Numerical Features

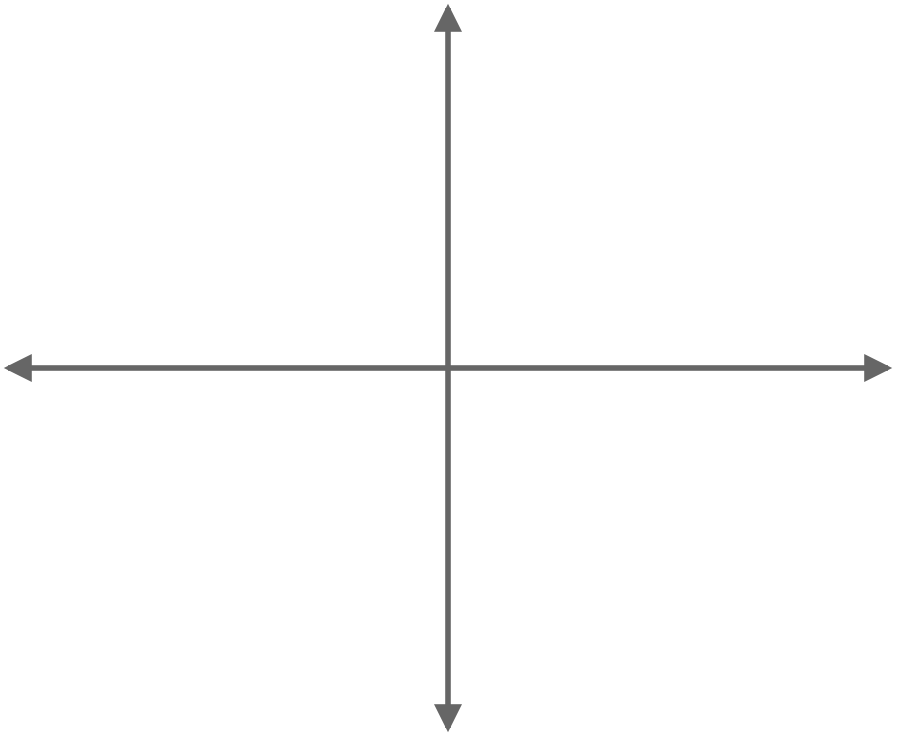
Continuous Feature: a numerical feature measured along a scale that can include decimal values.

- **Examples:** time of possession (34.23 minutes), yards per play

Discrete Feature: a numerical feature made of separate, countable values, usually whole numbers.

- **Examples:** points scored, turnovers, attendance

Calculating Similarity of Samples



Euclidean Distance: the straight-line distance between two feature vectors, found by squaring coordinate differences, adding them, and taking the square root.

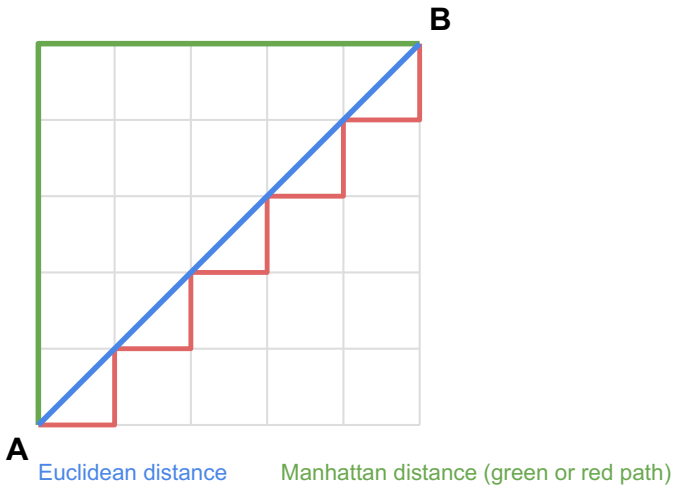
$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

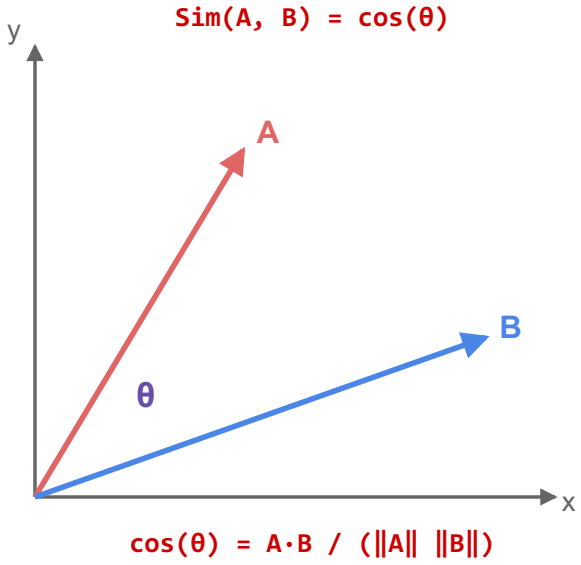
Other Distance Metrics

Metric: a rule for measuring distance or similarity between feature vectors.

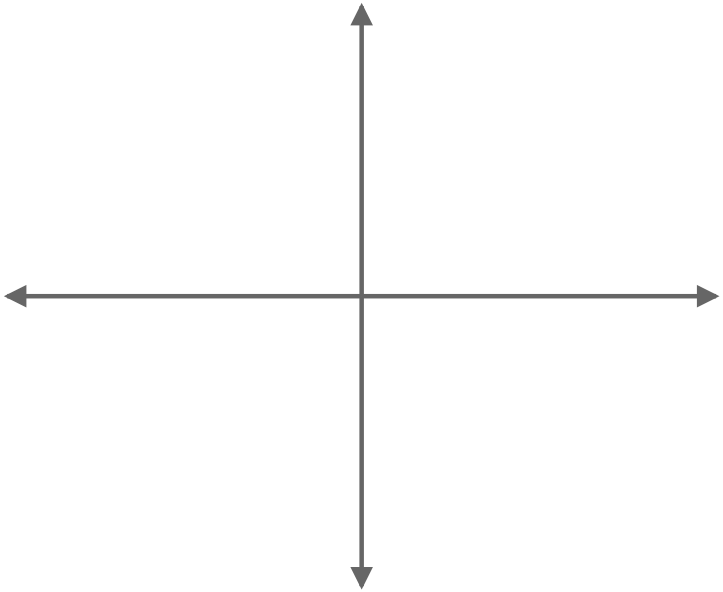
Manhattan Distance: the sum of the absolute coordinate differences between two feature vectors.

Cosine Similarity: a measure of how closely two feature vectors point in the same direction, regardless of their magnitude.





Dangers of Numeric Features



Scaling: changing numerical feature ranges so their units do not dominate a comparison or model.

Normalization: scaling features to a fixed range, usually from `0` to `1`.

Standardization: scaling features by subtracting the mean and dividing by the standard deviation.

Categorical Features

Categorical (nominal): Categories with no inherent order or ranking, just different labels. Assigning numbers to them would be arbitrary.

- **Examples:** the opponent, team colors, the stadium

Categorical (ordinal): Categories that have a meaningful order or ranking, but the "distance" between categories isn't necessarily equal or measurable.

- **Examples:** class year (freshman to senior), a poll ranking

One-hot Encoding

One-hot Encoding: representing a categorical value with a separate 0-or-1 **binary feature** for each category.

Label Encoding

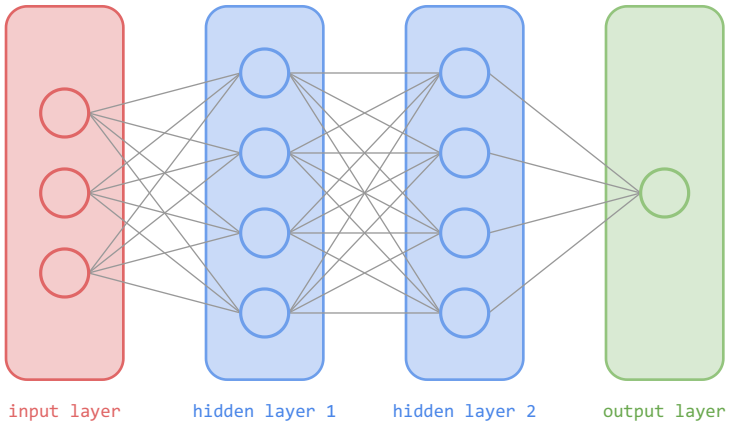
Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

Predicting ND Football Results

Opponent	ND home	Neutral	Attendance	ND pts	Opp. pts	Margin	Win
Alabama	0	1	80120	14	42	−28	0

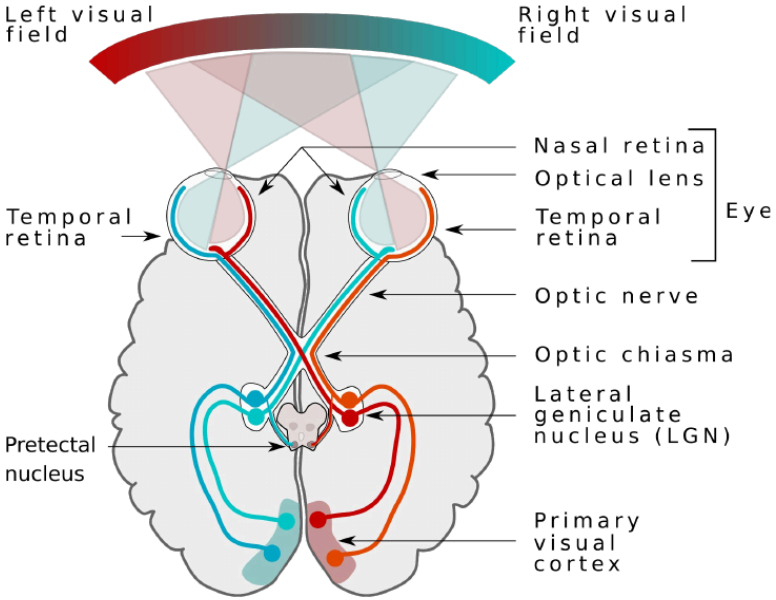


0

This requires changing the **representation** and **dimensionality** of our data

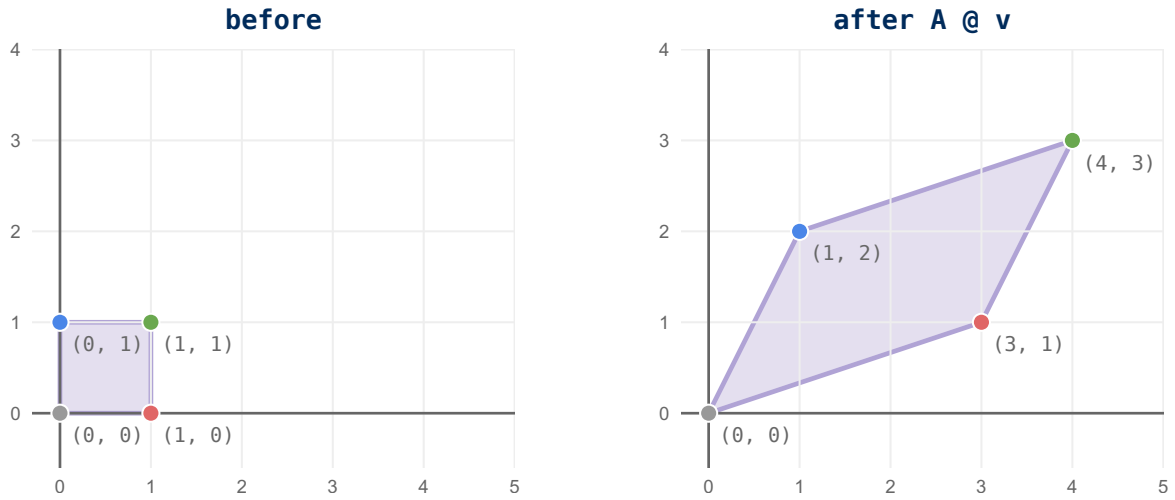
Representations in the Brain





Changing Dimensionality by Matrix Multiplication

We can think of multiplying matrices together as **creating better representations** of our data, eventually transforming from **input space** to **output (answer) space**



A maps every corner of the unit square: (0,0) → (0,0), (1,0) → (3,1), (0,1) → (1,2), (1,1) → (4,3)

$$\begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} @ \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

When we train a model, we could say that **our model is learning** how to **create better representations** through trial and error

Predicting ND Football Results

Opponent	ND home	Neutral	Attendance	ND pts	Opp. pts	Margin	Win
Alabama	0	1	80120	14	42	−28	0



Opponent	ND home	Neutral	Attendance
Alabama	0	1	80120



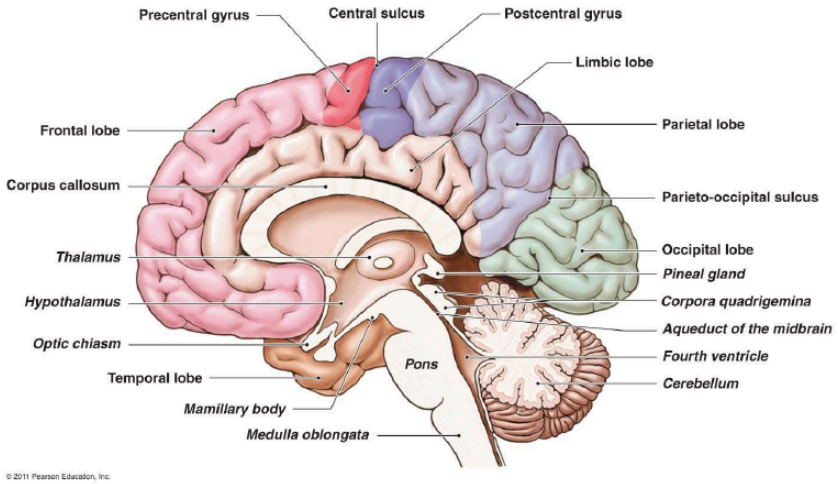
Target Leakage: information in an input **feature** that directly or indirectly reveals the **target**.

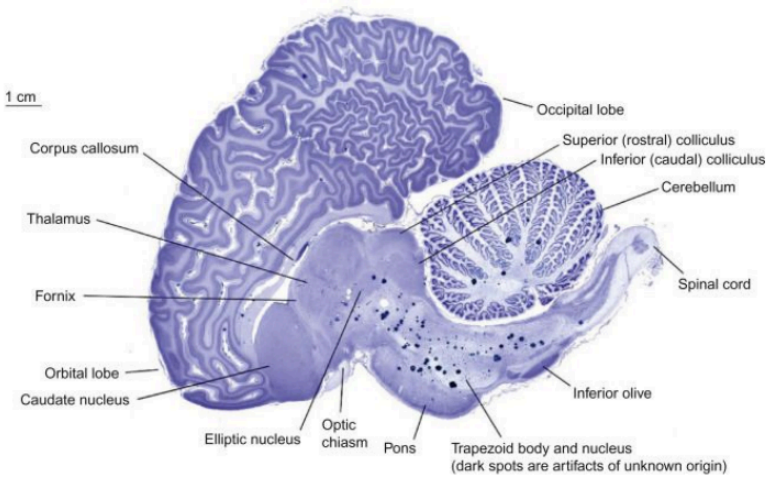
Predicting ND Football Results

Opponent	ND home	Neutral	Attendance
Alabama	0	1	80120



Anatomy of the Brain

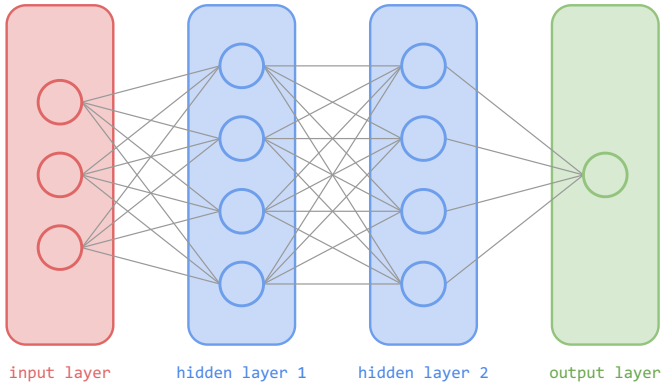


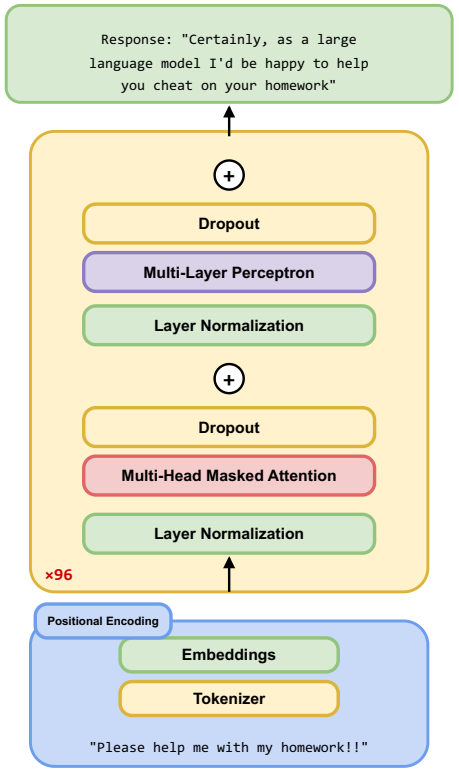


The **human brain** could be said to have a specific **architecture** which is **different from other animals**. The **brain's architecture** is one of the major ways in which we are different.

Architecture of a Model

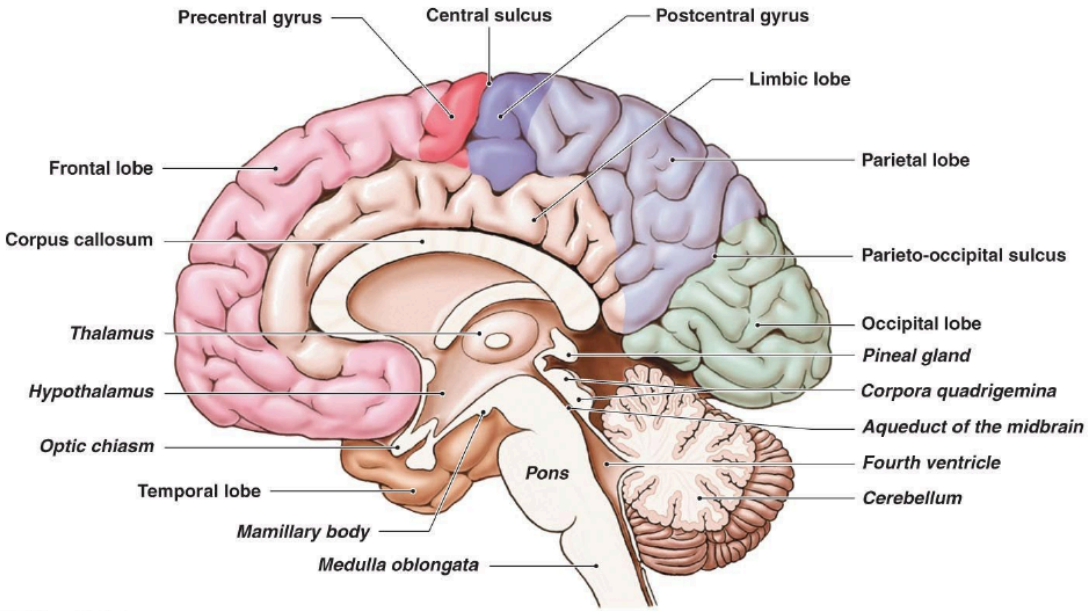
Every model has an **architecture**





A model's **architecture** can be thought of as the **specific set of matrix multiplications** the data goes through in the model

Architecture of the Human Brain



If we share the same **brain architecture**,

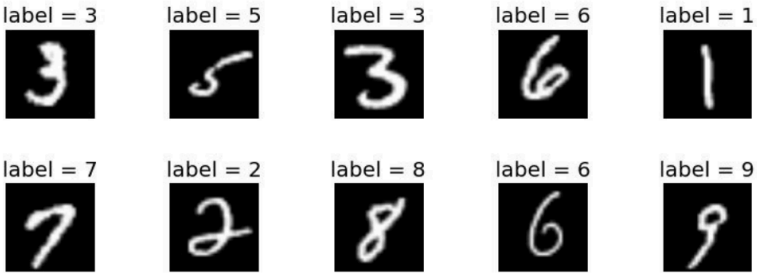
why are people so different?

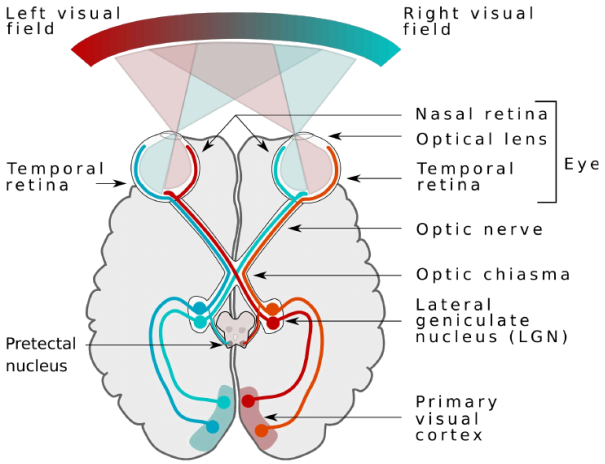
Programming: Google Colab

Google Colab is a service that provides hosted runtimes for **python notebooks**

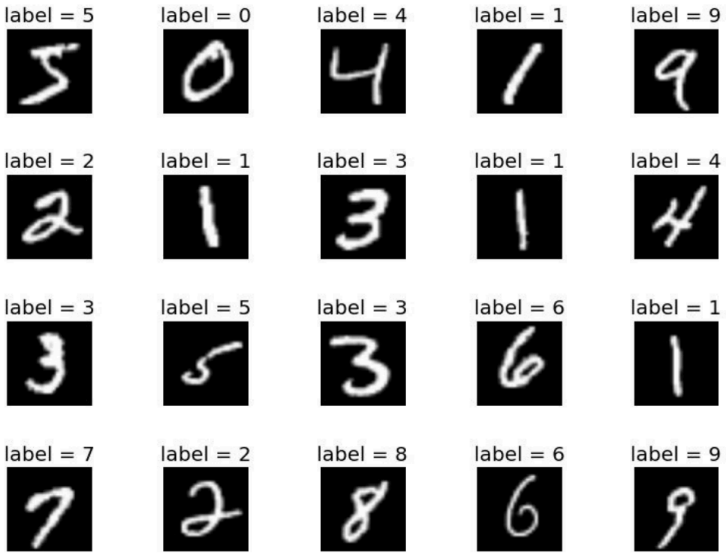
python notebooks are files that allow you to **mix text and code together** in a single document, and are one of the most common ways that **AI research and programming** is done.

Programming Day 01: Classifying Images



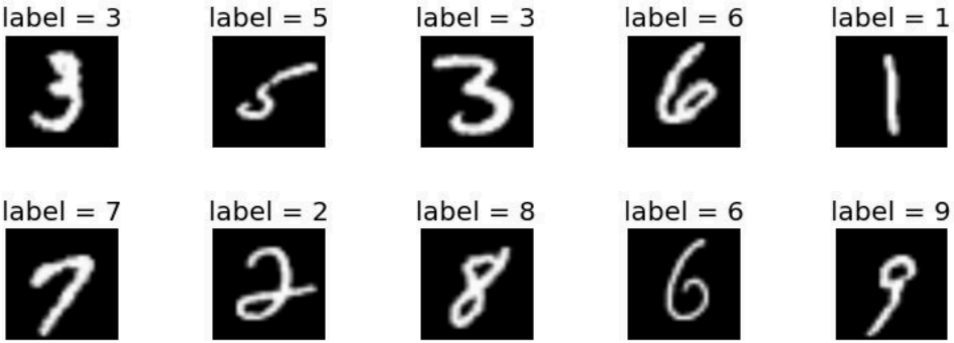


Classification Dataset: MNIST



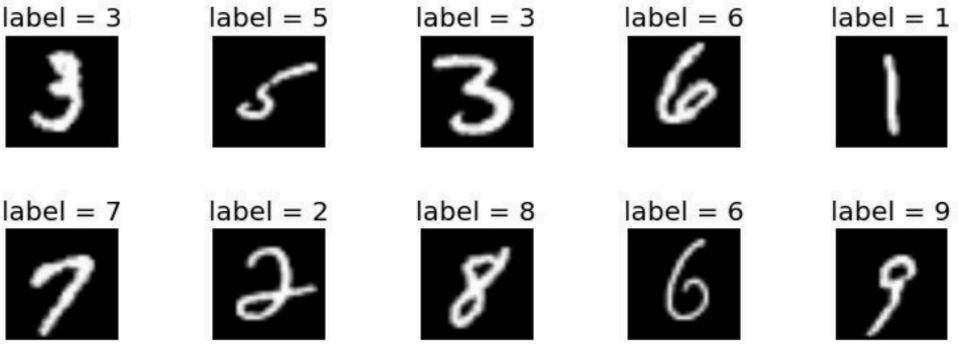
Learning: Supervised

We call it **supervised learning** when we have examples of the **target** in our **dataset**, because **we can tell our model if it guessed correctly or not**. We call these correct answers **ground truth**



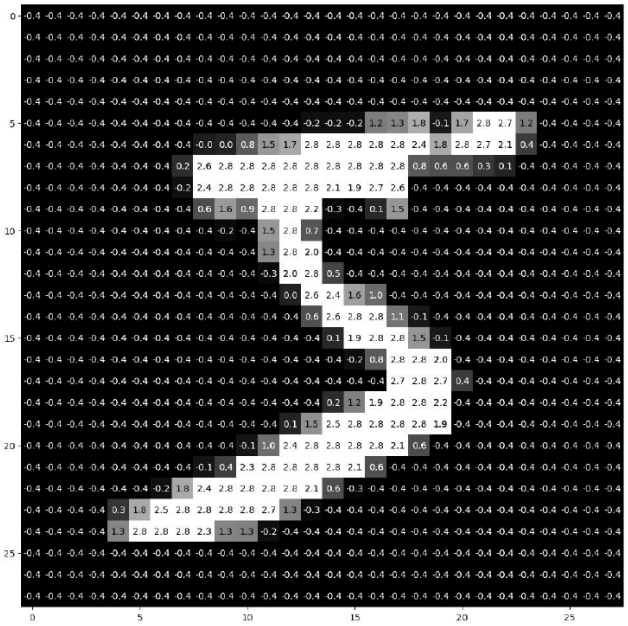
Task: Classification

In **machine learning**, a **classification** task is one in which we're trying to **predict a category or label (target)** given a **sample**.



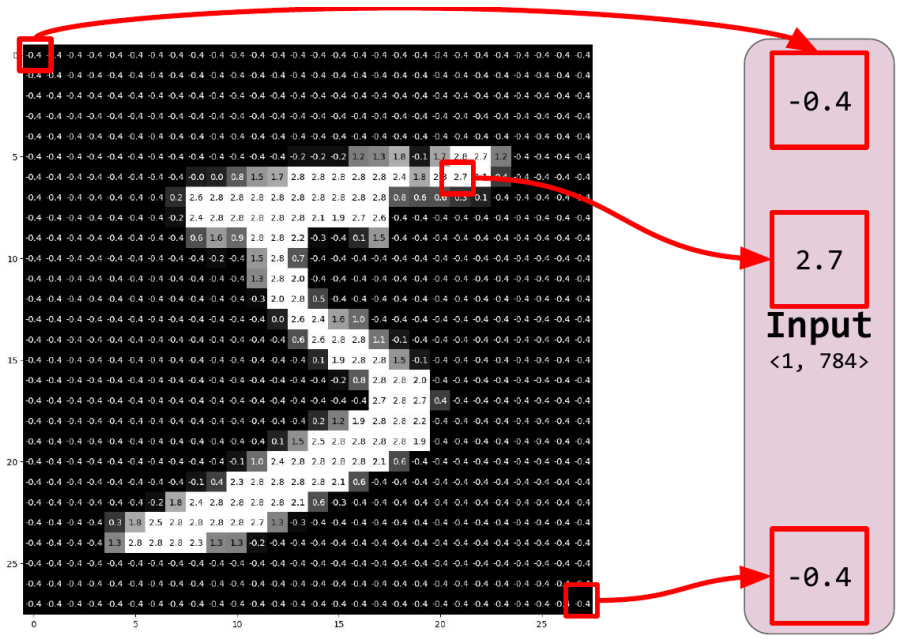
`MNIST` is a **Supervised** Multiclass **Classification Task**

Representing our Input



- **Input Data and Dimensionality:** a 28x28 image of a handwritten digit, 784 pixel values
- **Output Data and Dimensionality:** 10 scores, one for each digit 0 to 9

Representing our Input



Our images are **28x28** so we can **flatten** that into a **<1, 784>** dimensional **feature vector**

Representing an Output Prediction and the Ground Truth

We have `10` possible outputs **(targets/classes)** that we can produce. One for each number.



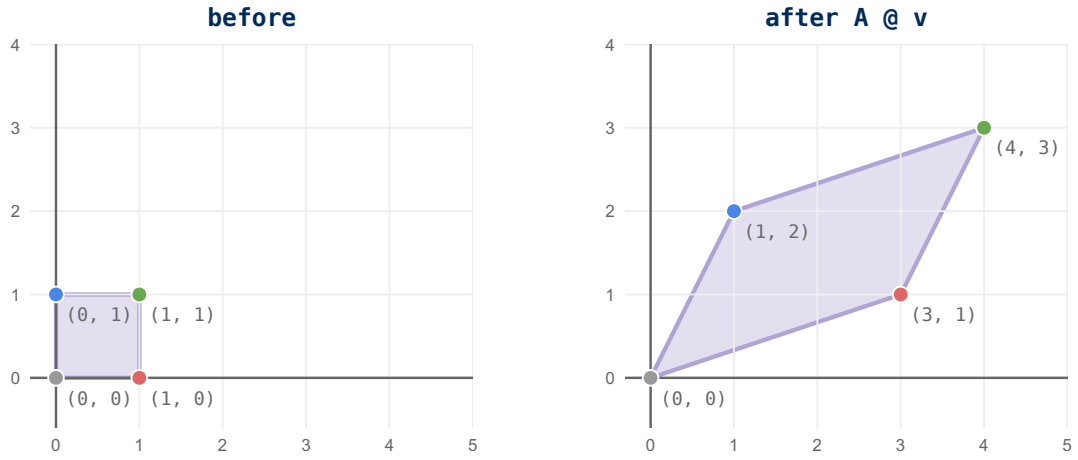
	0	1	2	3	4	5	6	7	8	9
$\hat{y}$ <1, 10>	0.0864	0.0956	0.1103	0.1796	0.0864	0.0703	0.0948	0.0925	0.0838	0.1003
y <1, 10>	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0

We return the **largest index value** as our prediction (`3` in this case), which we refer to as $\hat{y}$

Our **ground truth vector** is a **one-hot vector** with the **same size** as the output vector, with a `1` in the correct `index` and `0`s everywhere else. We refer to this vector as **y**

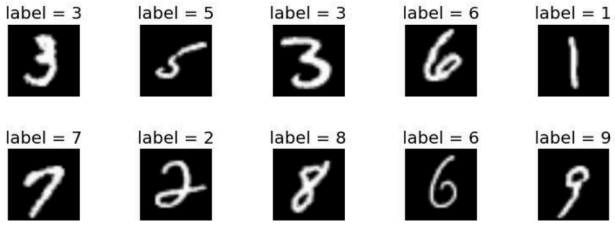
Changing Dimensionality by Matrix Multiplication

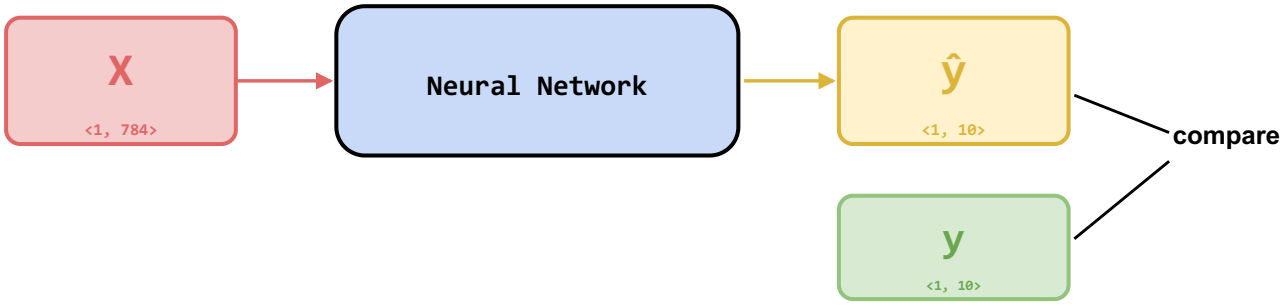
If our **input vector** is `<1, 784>` and our **output vector** is `<1, 10>` then we must somehow change the **dimensionality** of our data.



We can do this by multiplying matrices together to transform the data from **input space** `<1, 784>` to **output (answer) space** `<1, 10>`

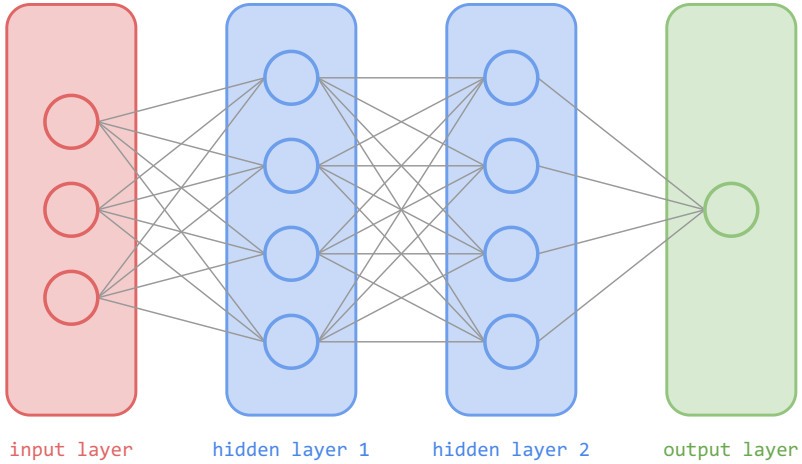
Producing an Output Prediction





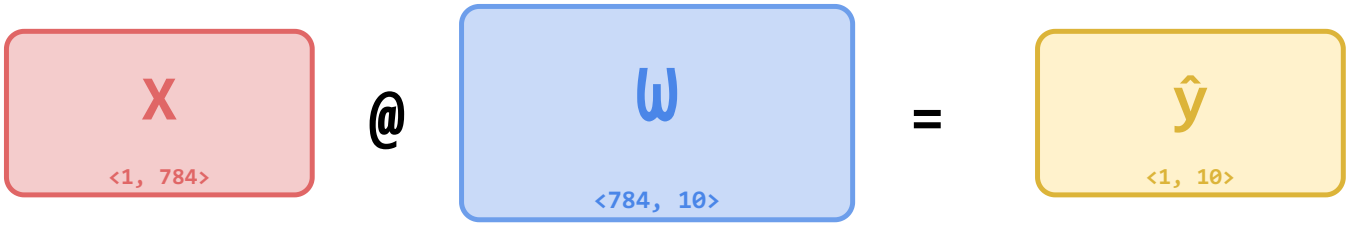
Architecture of a Model

Every model has an **architecture**



A model's **architecture** can be thought of as the **specific set of matrix multiplications** the data goes through in the model

Our Simple Model Architecture



Our model has a single layer, going from `784` dimensions down to `10` dimensions

Model Weights (and Biases)

tokens_embed.weight Vol 1 · p.9

offset 0

```
 0.046408 -0.128050 -0.015343  0.064112 -0.049744 -0.020974  0.128793  0.059280 -0.079039 -0.044810
 0.007333 -0.082801 -0.067706 -0.190251 -0.069751  0.061687 -0.023319 -0.067211 -0.040688  0.074863
-0.019345 -0.102572 -0.050788 -0.078921  0.042484  0.112099 -0.016808 -0.093247  0.165773 -0.014455
 0.159799  0.068638 -0.293205 -0.017096  0.130990 -0.002076 -0.032160 -0.074617 -0.075901  0.114713
 0.019735  0.040976  0.049918  0.068533 -0.018182  0.044382 -0.041536 -0.103330 -0.028750  0.033877
-0.105490  0.050557 -0.149537  0.261052 -0.066491 -0.161493 -0.038617 -0.190956  0.153980  0.108151
 0.048762 -0.113592 -0.146391 -0.047676 -0.085338 -0.071682 -0.038363  0.112355  0.029208  0.015777
 0.160551  0.022520 -0.126303  0.023652 -0.132281 -0.064657 -0.081478  0.022606 -0.025213  0.065771
 0.018249 -0.025612  0.048807  0.045837  0.052457  0.035308  0.046432 -0.089707 -0.051562 -0.111219
 0.023101 -0.106604 -0.012503  0.002402  0.003660 -0.002044  0.177684 -0.246009  0.139782 -0.058911
 0.046408 -0.128050 -0.015343  0.064112 -0.049744 -0.020974  0.128793  0.059280 -0.079039 -0.044810
 0.007333 -0.082801 -0.067706 -0.190251 -0.069751  0.061687 -0.023319 -0.067211 -0.040688  0.074863
-0.019345 -0.102572 -0.050788 -0.078921  0.042484  0.112099 -0.016808 -0.093247  0.165773 -0.014455
 0.159799  0.068638 -0.293205 -0.017096  0.130990 -0.002076 -0.032160 -0.074617 -0.075901  0.114713
 0.019735  0.040976  0.049918  0.068533 -0.018182  0.044382 -0.041536 -0.103330 -0.028750  0.033877
-0.105490  0.050557 -0.149537  0.261052 -0.066491 -0.161493 -0.038617 -0.190956  0.153980  0.108151
 0.048762 -0.113592 -0.146391 -0.047676 -0.085338 -0.071682 -0.038363  0.112355  0.029208  0.015777
 0.160551  0.022520 -0.126303  0.023652 -0.132281 -0.064657 -0.081478  0.022606 -0.025213  0.065771
 0.018249 -0.025612  0.048807  0.045837  0.052457  0.035308  0.046432 -0.089707 -0.051562 -0.111219
 0.023101 -0.106604 -0.012503  0.002402  0.003660 -0.002044  0.177684 -0.246009  0.139782 -0.058911
```

9

Python Library: PyTorch

`PyTorch` is **THE** library for doing **deep learning** and was created by **Meta** and is used by companies like **OpenAI**, **Tesla**, and **Microsoft**

However there are a few other options:

- `Keras`
  - Waymo, NASA, Intel
- `Tensorflow`
  - AirBnB, Uber, PayPal, Spotify
- `Jax`
  - Anthropic, Google Deepmind, xAI, Apple

Programming: PyTorch

- **Creating a tensor:** `torch.tensor()`, **its size:** `my_tensor.shape`
- **creating a weight matrix:** `torch.randn()`, **a bias:** `torch.zeros()`
- **matrix multiplication:** `@`

```python
import torch

# a stand-in for one 28x28 MNIST image
example_img = torch.rand(28, 28)
print('Example Image Shape:', example_img.shape)

x_flat = example_img.reshape(1, 784)
W = 0.01 * torch.randn(784, 10)
b = torch.zeros(10)

def forward(x_flat):
    return x_flat @ W + b

print(forward(x_flat).shape)
```

OUTPUT

```
Example Image Shape: torch.Size([28, 28])
torch.Size([1, 10])
```

Programming: PyTorch

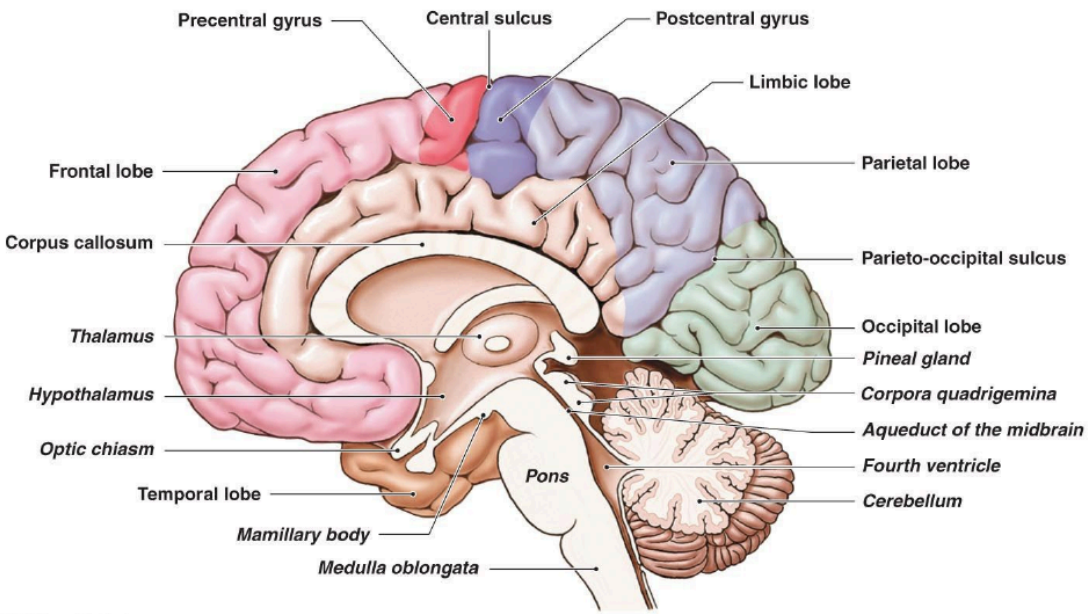
- **getting the largest value:** `torch.argmax()`
- **Creating a linear layer:** `torch.nn.Linear()`
- **creating a probability distribution:** `torch.nn.functional.softmax(output_vector)`

```
1  import torch
2
3  example_prediction_probs = [0.0864, 0.0956, 0.1103,
4                              0.1796, 0.0864, 0.0703,
5                              0.0948, 0.0925, 0.0838,
6                              0.1003]
7  probs = torch.tensor(example_prediction_probs)
8  print(torch.argmax(probs).item())
9
10 linear_layer = torch.nn.Linear(784, 10)
11 input_vector = torch.rand(1, 784)
12 output_vector = linear_layer(input_vector)
13 out = torch.nn.functional.softmax(output_vector, dim=1)
14 print(out.shape, round(out.sum().item(), 4))
```

OUTPUT

```
3
torch.Size([1, 10]) 1.0
```

Architecture of the Human Brain

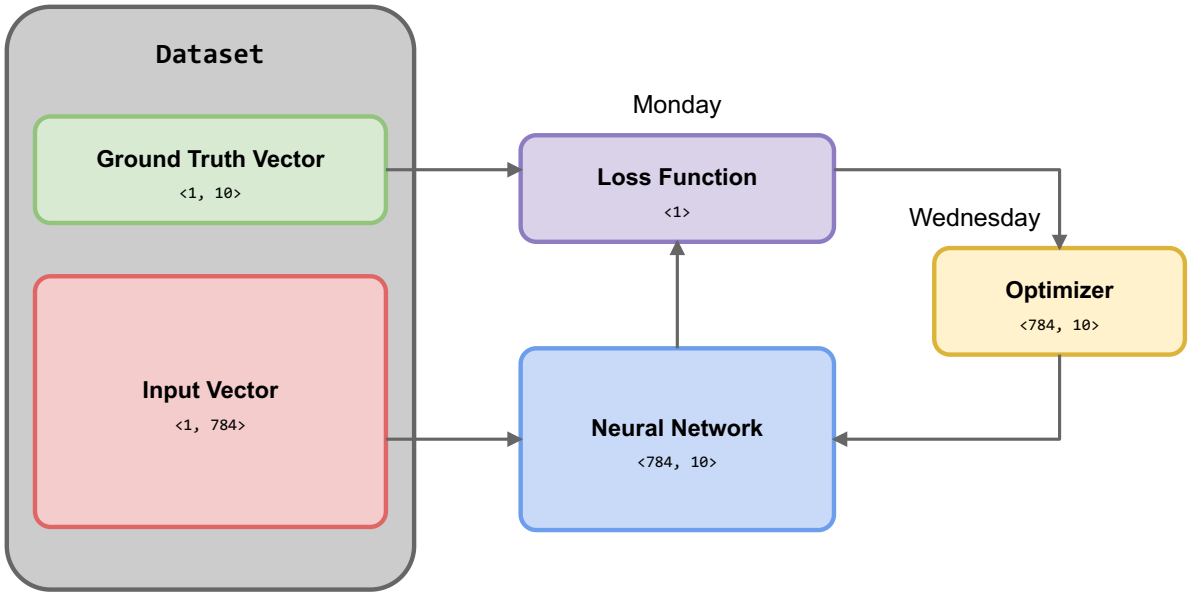


If we share the same **brain architecture**,

why are people so different?

Training a Model

So our model is bad. **To improve it, we can train the model**. This lets it practice over and over on the same set of data, ideally **improving its performance** over time! It does this **by updating its weights**.



Next Class: Loss Functions

P*: the true class distribution, P: the predicted class distribution

$$H(P^* \mid P) = -\sum_i P^*(i) \log P(i)$$